

Objective

The goal is to understand what British Airways customers are saying about their flights. By analyzing customer reviews, it's possible to identify patterns and areas for improvement. This information can help British Airways make changes to improve the flying experience and improve overall customer satisfaction. The focus is on understanding what customers like and dislike about different parts of their journey.

This includes looking at how happy customers are with different types of seats, flights, and routes operated by British Airways. The aim is to find out what's working well and what needs improvement, whether it's check-in, in-flight services, or baggage handling. By identifying these areas, British Airways can make targeted changes to deliver a better experience for its customers.

✓ Data Understanding

The dataset is a collection of information from British Airways customers. It includes their reviews, ratings, travel details, and suggestions. For instance, customers provide written feedback about their experiences, which is stored in the "reviews" section. They also give a rating out of five stars to show how satisfied they were.

The dataset also contains details about each customer's trip. This includes the type of seat they booked, such as Economy or Business class. Additionally, it notes the type of traveler they are, like a family or solo traveler. All this information helps paint a picture of each customer's experience with British Airways.

```
import pandas as pd
```

```
data = pd.read_csv("/content/British_Airway_Review.csv")  
data.head(10)
```



	reviews	date	country	seat_type	recommended	stars	route	type_of_travelle
0	 Trip Verified I had the most fantastic BA...	1st August 2023	Hong Kong	Business Class	yes	5	Heathrow to Las Vegas	Family Leisur
1	 Trip Verified Couldn't book in online. Ar...	31st July 2023	United Kingdom	Economy Class	no	3	Rome to Heathrow	Solo Leisur
2	 Trip Verified London Heathrow to Mumbai	31st July 2023	Iceland	Business Class	yes	3	Gatwick to Venice	Solo Leisur



data.shape



(2500, 8)

This dataset has 2500 rows and 8 columns

data.info()



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  -
0   reviews             2500 non-null   object
1   date                2500 non-null   object
2   country              2500 non-null   object
3   seat_type            2500 non-null   object
4   recommended          2500 non-null   object
5   stars                2500 non-null   int64
6   route                2500 non-null   object
7   type_of_traveller    2500 non-null   object
dtypes: int64(1), object(7)
```

memory usage: 156.4+ KB

✓ Data Preparation

To get the dataset ready for analysis, the first step is to clean and preprocess the information. This involves removing any unnecessary symbols or characters that don't add meaning to the text. For example, symbols like "✅ Trip Verified |" are taken out.

Next, the text is converted to lowercase to make it easier to work with. Any special characters, such as punctuation marks, are also removed. Additionally, any extra spaces between words are eliminated to tidy up the text. This helps create a consistent and organized dataset that's ready for further analysis.

```
data['cleaned_reviews'] = (  
    data['reviews']  
    .str.replace(r'✅ Trip Verified \|', '', regex=True)  
    .str.replace(r'^a-zA-Z0-9\s', '', regex=True)  
    .str.lower()  
    .str.strip()  
)
```

When working with dates, it's crucial to make sure they're all in the same format. This helps prevent errors and ensures everything runs smoothly. So, the date column is reviewed to catch any mistakes or inconsistencies.

If a date is missing or can't be read, it's replaced with "Unknown Date". This keeps the dataset tidy and reliable, even when some information is missing. By taking care of dates in this way, the dataset becomes more accurate and easier to understand.

```
data['date'] = pd.to_datetime(data['date'], errors='coerce', dayfirst=True).fillna("Unknown
```

To make the data easier to work with, some changes are made to the categorical variables. For instance, the "recommended" field is simplified to a yes or no answer, represented by a 1 or 0. This helps to clarify the data and make it more straightforward.

Two other fields, "seat_type" and "type_of_traveller", are transformed using a technique called one-hot encoding. This involves creating new fields that show which category each piece of data belongs to. For example, "seat_type" might be broken down into separate fields for Economy, Business, and other classes. This helps to make the data more flexible and easier to analyze.

Fill missing values in all other columns with "Unknown"

```
data.fillna("Unknown", inplace=True)
```

✓ Modeling

Sentiment Analysis

To understand how customers feel about their experiences, a sentiment analysis is done. This means categorizing reviews as Positive, Neutral, or Negative. This helps to see how customers feel.

The results are made easier to understand with visuals. One shows the overall breakdown of Positive, Neutral, and Negative reviews. Another shows how sentiments match up with star ratings. There's also a visualization that shows sentiment trends for different types of travelers and seat classes.

```
!pip install textblob
```

```
Requirement already satisfied: textblob in /usr/local/lib/python3.11/dist-packages (0.15
Requirement already satisfied: nltk>=3.9 in /usr/local/lib/python3.11/dist-packages (fr
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nl
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from r
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-package
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nlt
```

```
import textblob
from textblob import TextBlob

# Perform sentiment analysis on the cleaned reviews
data['sentiment'] = data['cleaned_reviews'].apply(lambda review: TextBlob(review).sentiment.

# Categorize sentiment into Positive, Neutral, and Negative
def categorize_sentiment(score):
    if score > 0.1:
        return "Positive"
    elif score < -0.1:
        return "Negative"
    else:
        return "Neutral"

data['sentiment'] = data['sentiment'].apply(categorize_sentiment)
```

```

import pandas as pd
import numpy as np
from textblob import TextBlob
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

# Set the visual style
plt.style.use('seaborn-v0_8')
sns.set_context("talk")
plt.rcParams['font.family'] = 'sans-serif'
plt.rcParams['font.sans-serif'] = ['Arial']

# Custom color palette - Modern and vibrant colors
colors = {
    'Positive': '#00b894', # Mint green
    'Neutral': '#fdbc6e', # Soft yellow
    'Negative': '#ff7675', # Coral red
    'background': '#f5f6fa' # Light background
}

# Create figure with a light background
plt.figure(figsize=(20, 10), facecolor=colors['background'])
plt.rcParams['axes.facecolor'] = colors['background']

# Create subplot grid
gs = plt.GridSpec(1, 2, width_ratios=[1, 1.2])
plt.subplots_adjust(wspace=0.3)

# 1. Enhanced Sentiment Distribution Plot
ax1 = plt.subplot(gs[0])
sentiment_order = ['Positive', 'Neutral', 'Negative']
palette = [colors[x] for x in sentiment_order]

# Calculate percentages
sentiment_counts = data['sentiment'].value_counts()
total_reviews = len(data)
sentiment_percentages = (sentiment_counts / total_reviews * 100).round(1)

# Create enhanced bar plot
bars = sns.barplot(
    x=sentiment_counts.reindex(sentiment_order).index,
    y=sentiment_counts.reindex(sentiment_order).values,
    palette=palette,
    ax=ax1
)

# Add total count and percentage labels
for i, (count, percentage) in enumerate(zip(sentiment_counts.reindex(sentiment_order),
                                          sentiment_percentages.reindex(sentiment_order))):
    # Add count on top of bar

```

```

    ax1.text(i, count, f'{int(count):,}',
             ha='center', va='bottom', fontsize=12, fontweight='bold',
             color='#2d3436')

    # Add percentage in middle of bar
    ax1.text(i, count/2, f'{percentage:.1f}%',
             ha='center', va='center', fontsize=12, fontweight='bold',
             color='white')

# Customize the plot
ax1.set_title('Sentiment Distribution\nAcross All Reviews',
              pad=20, fontsize=16, fontweight='bold')
ax1.set_xlabel('Sentiment Category', fontsize=14, labelpad=15)
ax1.set_ylabel('Number of Reviews', fontsize=14, labelpad=15)

# Add subtle grid only on y-axis
ax1.grid(axis='y', alpha=0.2, linestyle='--')
ax1.set_axisbelow(True)

# 2. Enhanced Traveler Type Analysis with Better Visualization
ax2 = plt.subplot(gs[1])

# Get the one-hot encoded columns for traveler types
traveler_type_columns = [col for col in data.columns if col.startswith('type_of_traveller_')]

# Melt the DataFrame to combine traveler type columns into a single column
melted_data = data.melt(id_vars=['sentiment'], value_vars=traveler_type_columns,
                        var_name='type_of_traveller', value_name='is_traveler_type')

# Filter for rows where is_traveler_type is 1 (indicating the traveler type)
filtered_data = melted_data[melted_data['is_traveler_type'] == 1]

# Create cross tabulation with percentages
cross_tab = pd.crosstab(filtered_data['type_of_traveller'], filtered_data['sentiment'],
                        normalize='index') * 100

# Create stacked bar plot with enhanced styling
cross_tab.plot(
    kind='bar',
    stacked=True,
    color=[colors['Positive'], colors['Neutral'], colors['Negative']],
    ax=ax2,
    width=0.8
)

# Customize the plot
ax2.set_title('Sentiment Distribution by Traveler Type',
              pad=20, fontsize=16, fontweight='bold')
ax2.set_xlabel('Traveler Type', fontsize=14, labelpad=15)
ax2.set_ylabel('Percentage of Reviews', fontsize=14, labelpad=15)

```

```
# Add percentage labels on bars
for c in ax2.containers:
    # Add labels only for segments > 5%
    labels = [f'{v:.1f}%' if v > 5 else '' for v in [h.get_height() for h in c]]
    ax2.bar_label(c, labels=labels, label_type='center',
                  fontsize=11, fontweight='bold', color='white')

# Enhance legend
ax2.legend(title='Sentiment',
           title_fontsize=12,
           fontsize=11,
           bbox_to_anchor=(1.05, 1),
           loc='upper left')

# Add subtle grid
ax2.grid(axis='y', alpha=0.2, linestyle='--')
ax2.set_axisbelow(True)

# Rotate x-axis labels for better readability
plt.setp(ax2.get_xticklabels(), rotation=45, ha='right')

# Add a main title to the entire figure
plt.suptitle('Customer Review Sentiment Analysis Dashboard',
             fontsize=20, fontweight='bold', y=1.05)

# Add footer with summary statistics
plt.figtext(0.02, -0.05,
            f'Total Reviews Analyzed: {total_reviews:,}',
            fontsize=10, style='italic')

# Adjust layout to prevent overlapping
plt.tight_layout()

# Show the plot
plt.show()
```

Passing ``palette`` without assigning ``hue`` is deprecated and will be removed in v0.14.0.

[illegible]

WARNING:matplotlib.font_manager.findfont: Generic family 'sans-serif' not found because

✓ Topic Modeling

Reviews often mention similar topics, and a technique called topic modeling helps identify these recurring themes. This process uses a special method to analyze the reviews and uncover common ideas.

The results are presented in a clear and easy-to-understand format. Keywords for each topic are shown in horizontal bar charts, making it simple to see what's being talked about. The most common themes are highlighted, providing valuable insights into what customers are saying.

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation as LDA
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns # For better color palettes

# 1. Create and Fit the Vectorizer
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(data)

# 2. Create and Fit the LDA model
n_topics = 3 # Choose the number of topics
lda = LDA(n_components=n_topics, random_state=42)
lda.fit(X)

# 3. Display Top Words for Each Topic (Improved Visualization)

# Set a professional color palette using Seaborn
# Use a diverging palette for topics or a sequential for word importance
palette = sns.color_palette("viridis", n_colors=10) # Example: "viridis", "cubehelix", "magma"

for index, topic in enumerate(lda.components_):
    top_indices = topic.argsort()[-10:]
    top_words = [vectorizer.get_feature_names_out()[i] for i in top_indices]
    top_values = topic[top_indices]

    plt.figure(figsize=(12, 8)) # Increased figure size

    # Create the horizontal bar plot
    bars = plt.barh(top_words, top_values, color=palette) # Use the selected palette

    # Add data labels for values (optional, but good for quick interpretation)
    for bar, value in zip(bars, top_values):
        plt.text(value + 0.01, bar.get_y() + bar.get_height()/2, f'{value:.2f}', va='center')

    plt.title(f"Topic {index + 1}: Top Words", fontsize=18, fontweight='bold', color="#333333")
    plt.xlabel("Word Importance", fontsize=14, color="#555555") # More descriptive label

```

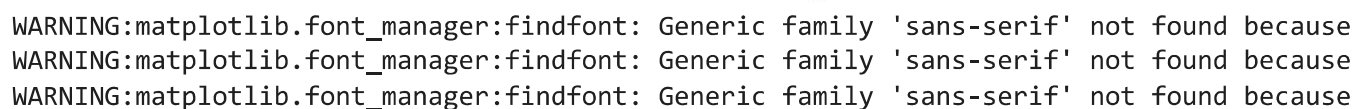
```
plt.ylabel("Words", fontsize=14, color="#555555")
plt.gca().invert_yaxis() # Top words at the top

# Improve aesthetics: grid, spines, ticks
plt.grid(axis='x', linestyle='--', alpha=0.7)
sns.despine(bottom=True, left=True) # Remove spines
plt.tick_params(axis='both', which='major', labelsize=12, color="#777777")

plt.tight_layout()
plt.show()
```

12/28

Topic 1: Top Words



This chart shows what people talk about most when reviewing flights. For example, 'date' and 'route' come up frequently, suggesting people often mention the timing and flight path. Things like 'class', 'stars', and 'reviews' having similar high scores means these topics – the travel class, star ratings, and written reviews themselves – are all really important to passengers too.

WARNING:matplotlib.font_manager.findfont: Generic family 'sans-serif' not found because

✓ Word Cloud

WARNING:matplotlib.font_manager.findfont: Generic family 'sans-serif' not found because

Highlight frequently used words in Positive and Negative reviews.

WARNING:matplotlib.font_manager.findfont: Generic family 'sans-serif' not found because

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud, STOPWORDS
import pandas as pd
import nltk # For natural language processing
from nltk.sentiment.vader import SentimentIntensityAnalyzer # For sentiment analysis
from nltk.corpus import stopwords
import re

# Download required NLTK resources (do this once)
nltk.download('vader_lexicon')
nltk.download('stopwords')

[➡] [nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True

WARNING:matplotlib.font_manager.findfont: Generic family 'sans-serif' not found because

# Initialize sentiment analyzer and stop words
analyzer = SentimentIntensityAnalyzer()
stop_words = set(stopwords.words('english'))

# Function to preprocess text (remove punctuation, numbers, etc.)
def preprocess_text(text):
    text = re.sub(r'[^\w\s]', '', text).lower() # Remove punctuation and convert to lowercase
    text = re.sub(r'\d+', '', text) # Remove numbers
    return text

# Function to filter words based on sentiment
def filter_words(text, sentiment_type, top_n=10):
    words = text.split()
    word_sentiments = {}

    for word in words:
        if word not in stop_words and len(word) > 2: # Remove stopwords and short words
            word = preprocess_text(word) # Preprocess the word
            if word: # Check if word is not empty after preprocessing
                sentiment_score = analyzer.polarity_scores(word)['compound']
```

```

if sentiment_type == 'positive' and sentiment_score > 0:
    word_sentiments[word] = sentiment_score
elif sentiment_type == 'negative' and sentiment_score < 0:
    word_sentiments[word] = sentiment_score

```

```

sorted_words = sorted(word_sentiments.items(), key=lambda item: item[1], reverse=True)
return [word for word, score in sorted_words[:top_n]]

```

```

# Separate reviews and filter words

```

```

positive_words = " ".join(filter_words(" ".join(data['reviews']), 'positive')) # Changed 'r
negative_words = " ".join(filter_words(" ".join(data['reviews']), 'negative')) # Changed 'r

```

```

# Function to create and display word cloud with customizations

```

```

def create_wordcloud(text, title, colormap, max_words=10):

```

```

    wordcloud = WordCloud(
        background_color='white',
        colormap=colormap,
        max_words=max_words,
        width=800,
        height=400,
        contour_width=2,
        contour_color='black',
        stopwords=STOPWORDS # Use NLTK's STOPWORDS
    ).generate(text)

```

```

plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.title(title, fontsize=16, fontweight='bold', color="#333333")
plt.tight_layout(pad=0)
plt.show()

```

```

# Create and display word clouds

```

```

create_wordcloud(positive_words, "Positive Reviews Word Cloud", 'Greens')
create_wordcloud(negative_words, "Negative Reviews Word Cloud", 'Reds')

```

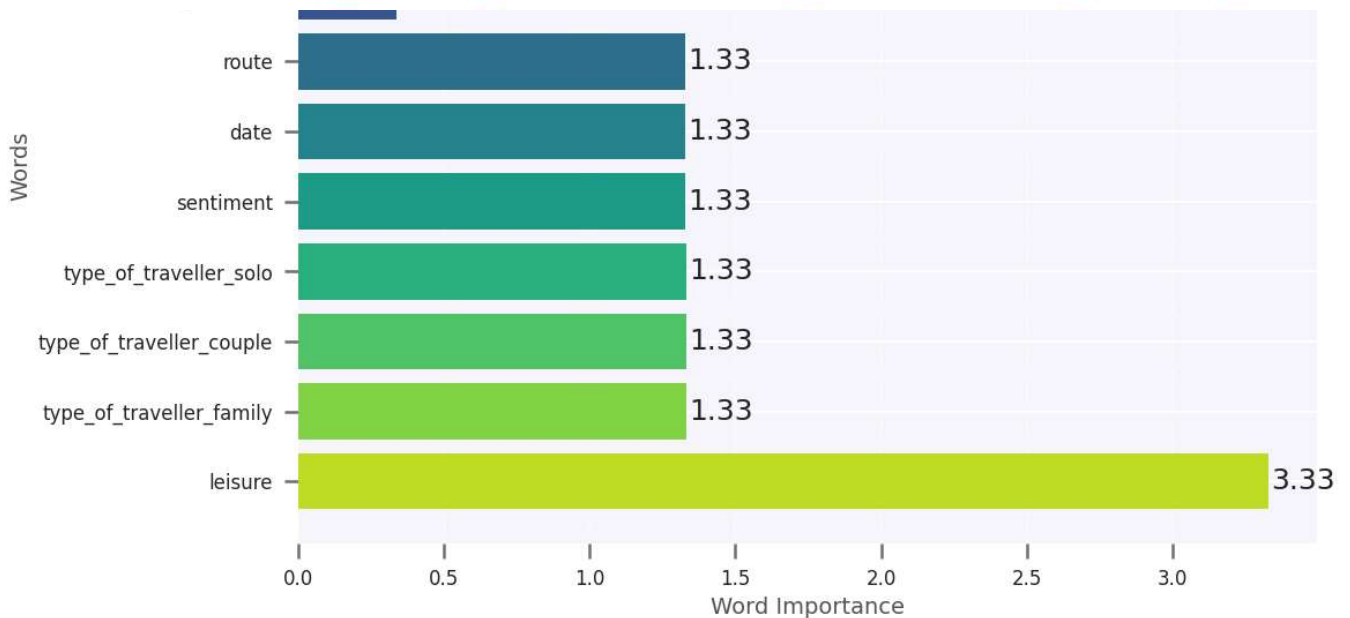
Positive Reviews Word Cloud



16/28

```
WARNING:matplotlib.font_manager:findfont: Generic family sans-serif not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
```

Negative Reviews Word Cloud



This word cloud shows the most common negative words people used in their reviews. Words like "hell", "suicide", and "terrorism" appearing so prominently suggests the experiences were extremely negative and upsetting.

```
# Sentiment Analysis (if 'sentiment' column is missing - as per your original code)
if 'sentiment' not in data.columns:
    data['sentiment'] = data['cleaned_reviews'].apply(lambda review: TextBlob(review).sentiment.polarity)
    data['sentiment'] = data['sentiment'].apply(categorize_sentiment)

# Ensure seat_type_columns is correctly defined
seat_type_columns = [col for col in data.columns if col.startswith('seat_type_')]
# If one-hot encoding for seat_type was not performed earlier, this list will be empty.
# If it's empty, use the original 'seat_type' column:
if not seat_type_columns:
    seat_type_columns = ['seat_type']

# Groupby and Plot
sentiment_by_seat = data.groupby(seat_type_columns + ['sentiment']).size().unstack(fill_value=0)

# Enhanced Plotting
fig, ax = plt.subplots(figsize=(14, 8))

# Custom colors - more visually appealing and aligned with sentiment connotations
colors = {"Positive": "#d63031", "Neutral": "#d3d3d3", "Negative": "#0b3991"} # Red for Positive, Grey for Neutral, Blue for Negative

sentiment_by_seat.plot(kind='bar', stacked=True, ax=ax, color=[colors['Negative'], colors['Neutral'], colors['Positive']])

plt.title("Sentiment Breakdown by Seat Type", fontsize=20, fontweight="bold", color="#333333")
plt.xlabel("Seat Type", fontsize=14, color="#555555") # Slightly darker axis labels
plt.ylabel("Number of Reviews", fontsize=14, color="#555555")
plt.xticks(rotation=45, fontsize=12, ha='right', color="#666666") # Angled, better aligned
plt.yticks(fontsize=12, color="#666666")
plt.legend(title="Sentiment", fontsize=12, title_fontsize=14, loc='upper left') # Legend position

# Add data labels - Iterating through each bar and stack
for container in ax.containers:
    for bar in container.patches:
        height = bar.get_height()
        if height > 0: # Don't add labels to zero height bars
            label_text = f'{int(height)}' # Format label as integer
            label_x = bar.get_x() + bar.get_width() / 2
            label_y = bar.get_y() + height / 2 # Position label in the middle of the segment
            ax.text(label_x, label_y, label_text, ha='center', va='center', color='white',
                    fontsize=10, fontweight='bold') # White, bold labels for contrast

# Improve grid and background for cleaner look
ax.yaxis.grid(True, linestyle='--', linewidth=0.7, color='lightgrey') # Vertical grid for y-axis
```

```
ax.xaxis.grid(False) # No vertical grid for x-axis
ax.set_facecolor('#f9f9f9') # Very light grey background for the plot area
fig.patch.set_facecolor('white') # White background for the figure

plt.tight_layout()
plt.show()
```

<https://colab.research.google.com/drive/1xHlZRrj5yHE6tll8cFKPfFj4zrdWP9LT?authuser=3#scrollTo=8Mqxp2nDgpU&printMode=true> 20/28

WARNING:matplotlib.font_manager.findfont: Generic family 'sans-serif' not found because

The rightmost seat type ('(True, False, False)') receives the most reviews, showing a generally positive sentiment trend, but also a notable portion of neutral and negative feedback. The first seat type ('(False, False, False)') is second in review volume with mixed sentiment, while the middle seat types receive significantly less feedback.

WARNING:matplotlib.font_manager:FontManager: Generic family 'sans-serif' not found because

1. Bar Chart for Seat Type Frequency

```
seat_type_counts = data['seat_type'].value_counts()
```

```
plt.figure(figsize=(10, 6))
```

```
seat_type_counts.plot(kind='bar', color=['#4c78a8', '#d63031', '#f58518', '#e45756', '#72b7b2'])
```

```
plt.title('Frequency of Seat Types', fontsize=16, fontweight='bold', color='#333333')
```

```
plt.xlabel('Seat Type', fontsize=12, color='#555555')
```

```
plt.ylabel('Number of Reviews', fontsize=12, color='#555555')
```

```
plt.xticks(rotation=45, ha='right', fontsize=10, color='#666666')
```

```
plt.yticks(fontsize=10, color='#666666')
```

```
plt.gca().spines['top'].set_visible(False) # Remove top spine
```

```
plt.gca().spines['right'].set_visible(False) # Remove right spine
```

Add data labels on top of bars

```
for i, v in enumerate(seat_type_counts):
```

```
    plt.text(i, v + 0.2, str(v), ha='center', va='bottom', fontsize=10, color='#333333')
```

```
plt.tight_layout()
```

```
plt.show()
```

23/28

```
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
WARNING:matplotlib.font_manager:findfont: Generic family 'sans-serif' not found because
```

The chart clearly indicates that Economy Class reviews are most prevalent in our dataset, significantly outpacing all other seat types. Business Class reviews are the next most frequent, while Premium Economy and First Class reviews are considerably less common.

Frequency of Seat Types

```
# 2. Pie Chart for Type of Traveller Frequency
```

```
import matplotlib.pyplot as plt
```

```
traveller_type_counts = data['type_of_traveller'].value_counts()
```

```
fig, ax = plt.subplots(figsize=(8, 8))
```

```
colors = ['#ff7f0e', '#2ca02c', '#9467bd', '#8c564b', '#e377c2'] # Different color palette
```

```
plt.pie(traveller_type_counts, labels=traveller_type_counts.index, colors=colors,
        autopct='%1.1f%%', startangle=90, wedgeprops={'linewidth': 0.5, 'edgecolor': 'white'}
        textprops={'fontsize': 10, 'color': '#333333'}) # Adjusted text properties
```

```
plt.title('Distribution of Traveller Types', fontsize=16, fontweight='bold', color='#333333')
```

```
plt.ylabel('') # Remove default y label
```

```
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
```

```
plt.tight_layout()
```

```
plt.show()
```

Distribution of Traveller Types



It breaks down who's writing these reviews based on their travel style. The biggest slice, by far, is from couples traveling for leisure. After that, it's pretty evenly split between business travelers, solo leisure travelers, and families on leisure trips. So, couples on vacation are our main reviewers, but we're getting feedback from all types of travelers.

✓ ML Modeling

```
# Encode the target variable (recommended)
data['recommended_encoded'] = data['recommended'].map({'yes': 1, 'no': 0})
```

```
data.head()
```



	reviews	date	country	seat_type	recommended	stars	route	type_of_travel
0	 Trip Verified I had the most fantastic BA...	2023-08-01 00:00:00	Hong Kong	Business Class	yes	5	Heathrow to Las Vegas	Family Leis
	 Trip Verified	2023-07						

```
# Drop the 'recommended' column from the dataset
data.drop(columns=['recommended'], inplace=True)
```

```
# Verify the column is dropped
print(data.head())
```



		reviews	date \
0	✓ Trip Verified	I had the most fantastic BA...	2023-08-01 00:00:00
1	✓ Trip Verified	Couldn't book in online. Ar...	2023-07-31 00:00:00
2	✓ Trip Verified	London Heathrow to Mumbai in...	2023-07-31 00:00:00
3	✓ Trip Verified	Keflavík, Iceland to London ...	2023-07-31 00:00:00
4	✓ Trip Verified	Terrible Experience with Bri...	Unknown Date

	country	seat_type	stars	route \
0	Hong Kong	Business Class	5	Heathrow to Las Vegas
1	United Kingdom	Economy Class	3	Rome to Heathrow
2	Iceland	Business Class	3	Gatwick to Venice
3	Iceland	Business Class	5	London to Luanda
4	Canada	Economy Class	5	Denver to Heathrow

	type_of_traveller	cleaned_reviews \
0	Family Leisure	i had the most fantastic ba flight today the c...
1	Solo Leisure	couldnt book in online arrived at check in to ...
2	Solo Leisure	london heathrow to mumbai in a boeing 7878 in ...
3	Couple Leisure	keflavk iceland to london heathrow on an a320 ...
4	Family Leisure	terrible experience with british airways i boo...

	recommended_encoded
0	1
1	0
2	1
3	1
4	0

```
# One-hot encode all categorical columns
categorical_columns = ['seat_type', 'type_of_traveller', 'route', 'country', 'sentiment']
for column in categorical_columns:
    if column in data.columns:
        data = pd.get_dummies(data, columns=[column], drop_first=True)
```

```
# Drop columns that are not needed for modeling
data.drop(columns=['reviews', 'cleaned_reviews', 'date'], inplace=True, errors='ignore')
```

```
# Define features (X) and target variable (y)
X = data.drop(columns=['recommended_encoded']) # Features
y = data['recommended_encoded'] # Target variable
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import warnings
warnings.filterwarnings("ignore")

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, st

# Print the size of training and testing sets
print(f"Training set size: {X_train.shape}")
print(f"Testing set size: {X_test.shape}")

```

```

⇒ Training set size: (2000, 82)
   Testing set size: (500, 82)

```

Data allocation: 2000 entries for training, 500 for testing. Each entry includes 82 features.

✓ Logistic Regression

A special type of analysis called Logistic Regression is used to understand binary choices, like yes or no answers. It calculates the probability of a certain outcome, such as whether a customer